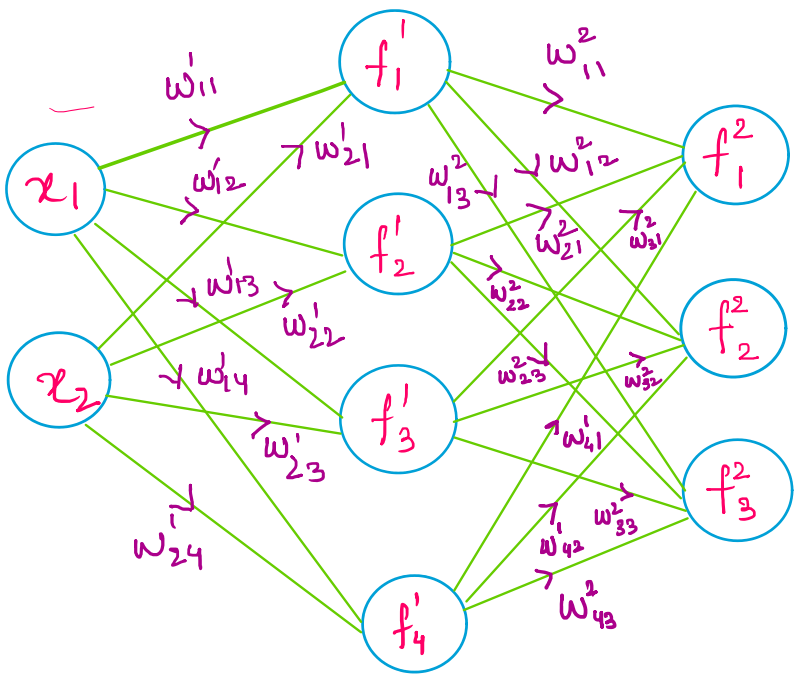
Neural Networks-3

★ Multi Layer Perceptron (MLP):-

NN formed by adding a layer of neurons in b/w an i/p and o/p layer. This intermediate layer is known as a hidden layer.
As the computation of these layers is not accessible from outside, hence these layers are called hidden layers.



weight matrix :

Layer-1

$$\begin{bmatrix} w_{11}^1 & w_{12}^1 & w_{13}^1 & w_{14}^1 \\ w_{21}^1 & w_{22}^1 & w_{23}^1 & w_{24}^1 \end{bmatrix}_{2\times 4}$$

Layer-2

$$\begin{bmatrix} w_{11}^2 & w_{12}^2 & w_{13}^2 \\ w_{21}^2 & w_{22}^2 & w_{23}^2 \\ w_{31}^2 & w_{32}^2 & w_{33}^2 \\ w_{41}^2 & w_{42}^2 & w_{43}^2 \end{bmatrix}_{4\times 3}$$

i/p layer | Hidden Layer (Layer-1) | O/P Layer (Layer-2)

i/p layer is not considered as a computation layer
Hidden and O/P layers are considered → neurons
computation ← activation function

NN can be made complex by increasing the no. of layers or by adding more neurons to one layer.

NN can be made complex by increasing the no. of layers or by adding more neurons to one layer.

* Activation Functions in case of Hidden Layers :-

① **Hyperbolic Tanget** :- It is a shifted version of sigmoid.

It works better than sigmoid.

I/P range → $-\infty$ to ∞

O/P range → -1 to $+1$

Formula :

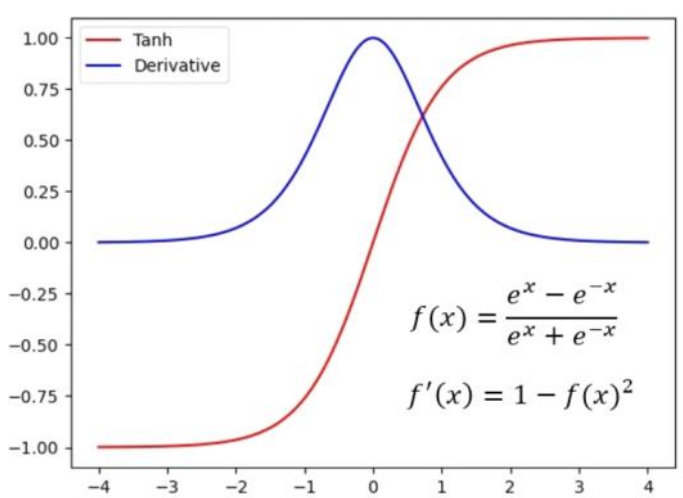
$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

& the derivative –

$$\tanh'(z) = 1 - \tanh^2(z)$$

The derivative lies b/w 0 and 1.

This function is used when we want O/P to lie in the range -1 to $+1$



Issue with sigmoid and tan h:-

Vanishing Gradients – Problem that occurs when gradients become too small [as the derivative lies b/w 0 and 1]

↓

Product will be very small

↓

weight update very slow → slow learning

* Ideal Activation :-

1. Differentiable
2. Non-Linear
3. Easy Calculation
4. Gradient ≥ 1 for big range of z

⇓

2. Rectified Linear Unit [ReLU] :-

An activation function for which –

$$ReLU = \begin{cases} z, & \text{if } z > 0 \\ 0, & \text{if } z \leq 0 \end{cases}$$

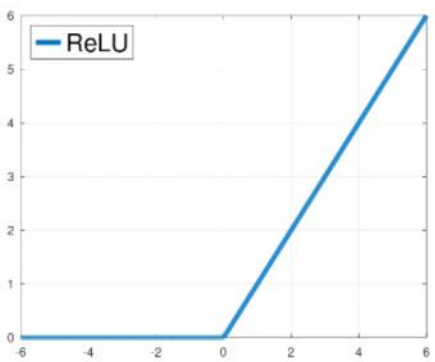
& the derivative –

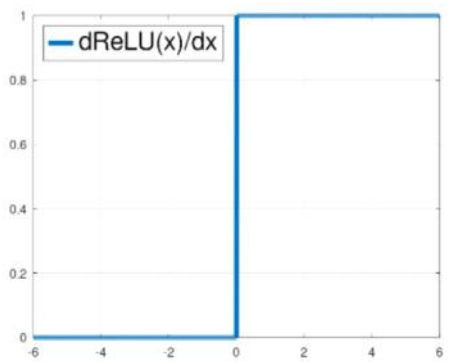
$$ReLU'(z) = \begin{cases} 1, & \text{if } z > 0 \\ 0, & \text{if } z < 0 \end{cases}$$

∵ it is non-differentiable at $z=0$ hence taking an approximation

$$\text{ReLU}'(z) = \begin{cases} 1, & \text{if } z>0 \\ 0, & \text{if } z \leq 0 \end{cases}$$ → approximation

Therefore, ReLU activation function is very fast to compute and is widely used in NN.





Though ReLU activation is widely used, but it comes with a problem.

Issue with ReLU :-

If anyone 1 term of the derivative while calculating the loss becomes 0, the entire term will become 0. This is known as dying ReLU.

↳ Dying of Neurons

Solution to this problem → Leaky ReLU

③ Leaky ReLU :-

This activation is similar to ReLU but comes with a

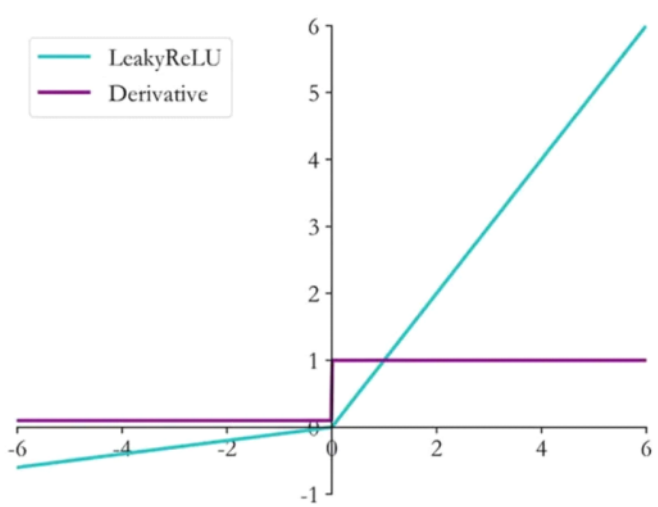
Small modification.
For $z<0$, a small gradient (α) is added.

$$\text{Leaky ReLU} = \begin{cases} z, & \text{if } z>0 \\ \alpha z, & \text{if } z\leq 0 \end{cases}$$

& its derivative –

$$\text{Leaky ReLU}'(z) = \begin{cases} 1, & \text{if } z>0 \\ \alpha, & \text{if } z\leq 0 \end{cases}$$

This solves the problem of dying ReLU. This variation of ReLU has a small +ve slope in the -ve area.
It enables backpropagation even for -ve i/p values
It is preferred when some neurons stop working / learning.



- The idea of Leaky ReLU can be extended further. Instead of

- The idea of Leaky ReLU can be extended further. Instead of multiplying z with a constant term, we can multiply it with a hyperparameter. This extension of leaky ReLU is known as Parametric ReLU.